

Практическая работа 1. Обеспечение качества программных продуктов

Управление качеством

Стандартизация в современном бизнесе и промышленности. Развитие мирового рынка привело к тому, что многие товары и услуги стали распространяться по всему миру, стали развиваться глобальные сервисы, в частности, телекоммуникационные, банковские. Для того чтобы устранить технические барьеры в промышленности, торговле и бизнесе, которые возникли вследствие того, что в разных странах для одних и тех же технологий и товаров действовали разнородные стандарты, стали создаваться национальные и международные комитеты по стандартизации. Самыми известными международными комитетами являются следующие.

1. ITU (International Telecommunication Union) – комитет образован в 1865 г. Штаб-квартира находится в Женеве (Швейцария), а ITU является частью ООН. Его основная задача – стандартизация телекоммуникационных протоколов и интерфейсов с целью поддержания и развития глобальной мировой телекоммуникационной сети. Самыми известными стандартами ITU являются:

- ISDN (цифровая телефонная связь, объединяющая телефонные сервисы и передачу данных);
- ADSL (широко известная модемная технология, позволяющая использовать телефонную линию для выхода в Интернет, не блокируя при этом обычного телефонного сервиса);
- OSI (модель открытого 7-уровневого сетевого протокола, на которой базируются все современные стандартные сетевые интерфейсы и протоколы; также является стандартом ISO);
- языки визуального проектирования телекоммуникационных систем, SDL и MSC, влившиеся позднее в UML.

Многие стандарты ITU переводятся на русский язык и преобразуются в российские стандарты в виде ГОСТов.

2. ISO (International Organization for Standardization) – организация создана в 1946 г. Цель – содействие развитию стандарти-

зации, а также смежных видов деятельности в мире с целью обеспечения международного обмена товарами и услугами, содействие и развитие сотрудничества в интеллектуальной, научно-технической и экономической областях. К настоящему времени создано около 17 000 стандартов в самых разных областях промышленности – продовольственные и иные товары, различное оборудование, банковские сервисы и т.д. Вот некоторые стандарты:

- серия стандартов ISO 9000. Направлены на стандартизацию качества товаров и услуг. Определение качества, определение системы поддержки качества на всех жизненных фазах изделия, товара, услуги (проектирование, разработка, коммерциализация, установка и обслуживание), описание процедур по улучшению деятельности компании, промышленного производства;
- ISO/IEC 90003:2004 – адаптация стандартов ISO 9000 к производству ПО в русле обеспечения качества в жизненном цикле ПО;
- ISO 9126:2001 – определение качественного ПО и различных атрибутов, описывающих это качество.

Многие стандарты ISO переводятся на русский язык и превращаются в российские стандарты в виде ГОСТов. Имеется много стандартов в области информационных технологий, а также несколько – в области программной инженерии. На соответствие стандартам ISO существует сертификация. В частности, компании сертифицируются на соответствие стандартам ISO 9000, то есть на качественный процесс разработки ПО.

3. ETSI (European Telecommunications Standards Institute) – организация создана в 1988 г., штаб-квартира находится в г. Софии (Антиполис) (Франция). Является независимой, некоммерческой организацией по стандартизации в телекоммуникационной промышленности (изготовители оборудования и операторы сети) в Европе. Самые известные стандарты – GSM, система профессиональной мобильной радиосвязи TETRA.

Остановимся теперь на ряде комитетов, непосредственно связанных с разработкой ПО.

1. SEI (Software Engineering Institute) – организация создана в 1984 год на базе университета Карнеги-Меллон в г. Питс-

бурге (США). Инициатор и главный спонсор – Министерство обороны США. Основная задача – стандартизация в области программной инженерии, выработка критериев для сертификации надежных и зрелых компаний (что в первую очередь интересует Минобороны США для выполнения его заказов). Самые известные продукты – стандарт CMM, CMMI, разработки в области семейства программных продуктов (product lines). Эти продукты шагнули далеко за пределы военных разработок США, их использование и развитие стали международной деятельностью. Некоторые продукты SEI стандартизованы также ISO. На соответствие CMM/CMMI проводится сертификация.

2. IEEE (Institute of Electrical and Electronics Engineers) – организация создана в 1963 г. Ведет историю с конца XIX века, в контексте промышленной стандартизации в США. IEEE является международной некоммерческой ассоциацией специалистов в области техники, мировой лидер в области разработки стандартов по радиоэлектронике и электротехнике. Штаб-квартира в США, существуют многочисленные подразделения в разных странах, включая Россию. IEEE издаёт третью часть мировой технической литературы, касающейся применения радиоэлектроники, компьютеров, систем управления, электротехники, в том числе (январь 2008) 102 реферируемых научных журнала и 36 отраслевых журналов для специалистов, проводит в год более 300 крупных конференций, принимала участие в разработке около 900 действующих стандартов.

3. OMG (Object Management Group) – организация создана в 1989 г. группой американских IT-компаний (в том числе Hewlett Packard, Sun Microsystems, Canon). В настоящий момент включает около 800 компаний членов. Основное направление – разработка и продвижение объектно ориентированных технологий и стандартов, в том числе для создания платформенно-независимых программных приложений уровня предприятий. Известные стандарты CORBA, UML, MDA.

Все эти комитеты и организации включают программную инженерию в сферу своей деятельности, сотрудничают, выпускают совместные стандарты, используют наработки друг друга и т.д.

Стандартизация качества. С точки зрения тестирования ПО нас интересует в этих стандартах стандартизация качества (как контекст тестирования) – сначала выпускаемой продукции, а потом и процессов по ее разработке. Следует отметить, что качественного результата не создать без качественного процесса. Обеспечение качества является более общим контекстом для тестирования.

Качество продукта или сервиса, предназначенного потребителю, определяется в стандарте ISO 9000:2005 как степень соответствия его характеристик требованиям обязательным или подразумеваемым.

Методы обеспечения качества ПО. Основные способы контроля качества, используемые на практике при разработке ПО:

- *наладка качественного процесса*, другими словами, совершенствование процесса. Для комплексного улучшения процессов в компании компаниями-разработчиками ПО используются стандарты CMM/CMMI, а также по стандартам серии ISO 9000 (с последующей официальной сертификацией);
- *формальные методы* – использование математических формализмов для доказательства корректности, спецификации, проверки формального соответствия, автоматической генерации (доказательство правильности работы программ, проверка на моделях определенных свойств, статический анализ кода по дереву разбора программы, модельно ориентированное тестирование, автоматическая генерация тестов и тестового окружения по формальным спецификациям требований к системе). На практике применяются ограниченно из-за необходимости серьезной математической подготовки пользователей, сложности в освоении, большой работы по развертыванию. Эффективны для систем, имеющих повышенные требования к надежности. Также имеются случаи эффективного использования средств, основанных на этих методах, в руках высококвалифицированных специалистов;
- *исследование и анализ динамических свойств ПО.* Например, широко используется профилирование – исследование использования системой памяти, ее быстродействие и другие характеристики путем запуска и непосредственных наблю-

дений в виде графиков, отчетов. В частности, этот подход используется при распараллеливании программ, при поиске «узких» мест;

- *обеспечение качества кода*. Сюда относится целый комплекс различных мероприятий и методов. Разработка стандартов оформления кода в проекте и контроль за соблюдением этих стандартов. Сюда входят правила на создание идентификаторов переменных, методов и имен классов, на оформление комментариев, правила использования стандартных для проекта библиотек. Регулярный рефакторинг для предотвращения образования из кода «вермишели». Существует тенденция ухудшения структуры кода при внесении в него новой функциональности, исправления ошибок. Различные варианты инспекции кода, например, техника peer code review. Последняя заключается в том, что код каждого участника проекта (выборочно) читается и обсуждается на специальных встречах (code review meetings), и делается это регулярно. Практика показывает, что в целом код улучшается;
- *тестирование*. Самый распространенный способ контроля качества ПО, представленный фактически в каждом программном проекте.

Качество программного обеспечения

Качество программного продукта определяется по нескольким критериям. Качественный программный продукт должен отвечать функциональным и нефункциональным требованиям, в соответствии с которыми он создавался, иметь ценность для бизнеса, отвечать ожиданиям пользователей.

В жизненном цикле управления приложениями качество должно отслеживаться на всех этапах жизненного цикла ПО. Оно начинает формироваться с определения необходимых требований. При задании требований необходимо указывать желаемую функциональность и способы проверки её достижения.

Качественный программный продукт должен обладать высоким потребительским качеством, независимо от области применения: внутреннее использование разработчиком, бизнес, наука и

образование, медицина, коммерческие продажи, социальная сфера, развлечения, веб и др. Для пользователя программный продукт должен удовлетворять определенному уровню его потребностей.

Важным аспектом создания качественного ПО является обеспечение нефункциональных требований, таких, как: удобство в эксплуатации, надежность, производительность, защищенность, удобство сопровождения. Надежность ПО определяет способность без сбоев выполнять заданные функции в заданных условиях и в течение заданного отрезка времени. Производительность характеризуется временем выполнения заданных транзакций или длительных операций. Защищенность определяет степень безопасности системы от повреждений, утраты, несанкционированного доступа и преступной деятельности. Удобство сопровождения определяет легкость, с которой обслуживается продукт в плане простоты исправления дефектов, внесения корректив для соответствия новым требованиям, управления измененной средой.

Управление жизненным циклом программного продукта помогает разработчикам целенаправленно добиваться создания качественного ПО, избегать потерь времени на переделку, повторное проектирование и перепрограммирование ПО.

Тестирование

Тестирование – это проверка соответствия между реальным поведением программы и ее ожидаемым поведением в специально заданных, искусственных условиях.

Ожидаемое поведение программы. Исходной информацией для тестирования является знание о том, как система должна себя вести, то есть требования к ней или к ее отдельной части. Самым распространенным способом тестирования является тестирование методом *черного ящика*, то есть когда реализация системы недоступна тестирующему, а тестируется только ее интерфейс. Часто это закрепляется и организацией коллектива – тестирующие оказываются отдельными сотрудниками и в некоторых компаниях они даже принципиально не общаются с разработчиками, чтобы минимально знать реализационные детали и максимально полно выступить в роли проверяющей инстанции. Существует тестиро-

вание методом *белого ящика*, когда код программ доступен тестировщику и используется в качестве источника информации о системе. Его схема представлена на рис. 8.1.

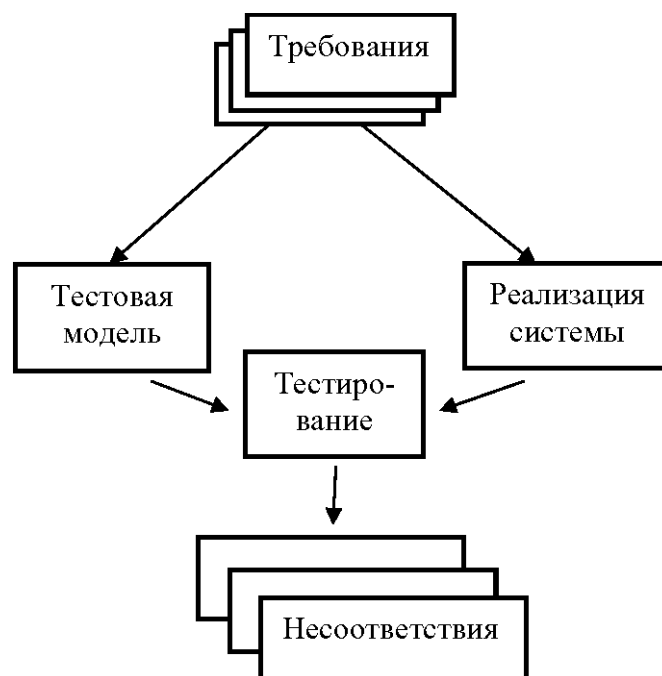


Рисунок 8.1 – Тестирование методом белого ящика

На этом рисунке видно, что на основе требований к системе создается реализация и тестовая модель системы. Тестирование есть сопоставление двух этих представлений с целью выявить их несоответствия. Чем независимее друг от друга будут эти представления, тем более информативно их сопоставление. Иначе если тестировщики существенно используют информацию о реализации системы при составлении тестов, то они могут невольно внести в тесты ошибки реализации. Найденное при тестировании несоответствие – это еще не ошибка, поскольку сами тестировщики могли неправильно понять требования, в тестах и средствах тестирования могли быть ошибки.

Данный подход закрепляется также и в организации коллективов программистов – тестировщики, как правило, отделены от разработчиков. Это разные люди, несовместимые роли в MSF.

Тесты могут быть «ручными» и автоматизированными. «Ручной» тест – это последовательность действий тестировщика, которую он (или разработчик) может воспроизвести, и ошибка

произойдет. Как правило, в средствах контроля ошибками такие последовательности действий содержатся в описании ошибки. Автоматический тест – это некоторая программа, которая воздействует на систему и проверяет то или иное ее свойство. Автоматический тест, по сравнению с «ручным», можно легко воспроизводить без участия человека. Можно создавать наборы тестов и прогонять их часто, например, в режиме регрессионного тестирования. Кроме того, автоматические тесты можно генерировать по более высокоуровневым спецификациям, например, по формально описанным требованиям к системе. А, например, тесты для компиляторов можно генерировать по формальному описанию языка программирования.

Таким образом, преимущества автоматических тестов перед «ручными» очевидны. В то же время автоматическому тестированию присущ ряд проблем.

Во-первых, для того, чтобы тесты автоматически запускать, нужны соответствующие программные продукты, которые также являются неотъемлемой частью специально заданных, искусственных условий. Их будем называть *инструментами тестирования*. В их задачу входит запуск теста на системе, «прогон» целого пакета тестов, а также анализ получившихся результатов и их обработка.

Кроме того, немаловажной задачей инструментов тестирования является обеспечение доступа теста к системе через некоторый ее интерфейс. Доступ к системе может оказаться затруднительным, например, в силу политических обстоятельств, когда сторонними разработчиками делается подсистема некоторой стратегической системы, и доступ к этой объемлющей системе у разработчиков сильно ограничен. Или в силу аппаратных ограничений – трудно «залезть» на «железку», где работает целевой код системы.

Кроме того, часто трудно «бесшовно» тестировать систему, оказывая на нее минимальное воздействие и добираясь при этом до всех аспектов ее функционирования. В целом настройка и развертка готовых, сторонних тестовых инструментов часто оказываются дорогостоящей и непростой задачей. Разработка своих собственных тестовых инструментов также не проста.

Во-вторых, часто возникает проблема ресурсов для автоматического тестирования. Особенно при автоматической генерации тестов: часто есть возможность автоматически сгенерировать очень большое количество тестов, так что если их еще выполнять регулярно, в режиме непрерывной интеграции, то не хватит имеющихся системных ресурсов. При этом качество тестирования может оказаться неудовлетворительным — ошибки находятся редко или вообще не находятся. Дело в том, что количество всех возможных состояний программной системы очень велико, и тестирование не может покрыть их все. На практике, в реальных проектах, определяют *критерии тестирования*, которые определяют ту «планку» качества, которую необходимо достичь в этом проекте. Ведь хорошее качество стоит дорого и очевидно, что разное ПО имеет разное качество, например, система управления ядерным реактором и текстовый редактор. На практике часто качество ПО определяется бюджетом проекта по его разработке. Далее, в силу ограниченности ресурсов на тестирование часто целесообразно бывает определить те аспекты ПО, которые наиболее важны — как для общей работоспособности системы, так и для заказчика. Например, при тестировании веб-приложения, предоставляющего услугу по созданию объявлений о продаже недвижимости, такими критериями были:

- правильность переходов сложного мастера — в частности, в связи с возможностью переходов назад;
- целостность введенных пользователем данных о создаваемых объявлениях.

Наконец, кроме ограничения количества тестов их отбора, важным является их прогон на некоторых входных данных. Часто здесь применяют принцип *факторизации* — множество всех возможных входных значений разбивают на значимые с точки зрения тестирования классы и «прогоняют» тесты не на всех возможных входных значениях, а берут по одному набору значений из каждого класса. Например, тестируют некоторую функцию системы на ее граничные значения — очень большие значения параметров, очень маленькие и пр. Часто факторизацию удобно делать, исходя из требований к данной функции, также бывает полезно посмотреть на ее реализацию и «пройтись» тестами по раз-

ным ее логическим веткам (порождаемым, например, условными операторами).

Виды тестирования. Приведем основные следующие виды тестирования [11, 13]:

- *модульное тестирование* – тестируется отдельный модуль, в отрыве от остальной системы. Самый распространенный случай применения – тестирование модуля самим разработчиком, проверка того, что отдельные модули, классы, методы делают действительно то, что от них ожидается. Различные среды разработки широко поддерживают средства модульного тестирования – например, популярная свободно распространяемая библиотека для Visual Studio NUnit, JUnit для Java и т.д. Созданные разработчиком модульные тесты часто включаются в пакет регрессионных тестов и таким образом, могут запускаться многократно;
- *интеграционное тестирование* – два и более компонентов тестируются на совместимость. Это очень важный вид тестирования, поскольку разные компоненты могут создаваться разными людьми, в разное время, на разных технологиях. Этот вид тестирования, безусловно, должен применяться самими программистами, чтобы, как минимум, удостовериться, что все живет вместе в первом приближении. Далее тонкости интеграции могут исследовать тестировщики. Необходимо отметить, что такого рода ошибки – «ошибки на стыках» – непросто обнаруживать и устранять. Во время разработки все компоненты все вместе не готовы, интеграция откладывается, а в конце обнаруживаются трудные ошибки (в том смысле, что их устранение требует существенной работы). Здесь выходом является ранняя интеграция системы и в дальнейшем использование практики постоянной интеграции;
- *системное тестирование* – это тестирование всей системы в целом, как правило, через ее пользовательский интерфейс. При этом тестировщики, менеджеры и разработчики акцентируются на том, как ПО выглядит и работает в целом, удобно ли оно, удовлетворяет ли оно ожиданиям заказчика. При этом могут открываться различные дефекты, такие как

неудобство в использовании тех или иных функций, забытые или «скудно» понятые требования;

- *регрессионное тестирование* – тестирование системы в процессе ее разработки и сопровождение на нерегресс. То есть проверяется, что изменения системы не ухудшили уже существующей функциональности. Для этого создаются пакеты регрессионных тестов, которые запускаются с определенной периодичностью – например, в пакетном режиме, связанные с процедурой постоянной интеграции;
- *нагрузочное тестирование* – тестирование системы на корректную работу с большими объемами данных. Например, проверка баз данных на корректную обработку большого (предельного) объема записей, исследование поведения серверного ПО при большом количестве клиентских соединений, эксперименты с предельным трафиком для сетевых и телекоммуникационных систем, одновременное открытие большого числа файлов, проектов;
- *стрессовое тестирование* – тестирование системы на устойчивость к непредвиденным ситуациям. Этот вид тестирования нужен далеко не для каждой системы, так как подразумевает высокую планку качества;
- *приемочное тестирование* – тестирование, выполняемое при приемке системы заказчиков. Более того, различные стандарты часто включают в себя наборы приемочных тестов. Например, существует большой пакет тестов, поддерживаемых компанией Sun Microsystems, которые обязательны для прогона для всех новых реализаций Java-машины. Считается, что только после того как все эти тесты успешно проходят, новая реализация вправе называться Java.

Устранение ошибок

Между программистами и тестировщиками необходим специальный интерфейс общения. Ведь ошибок находится много, их исправление требует времени, за их исправление отвечают разработчики, а тестировщики должны удостовериться, что они действительно исправлены. Кроме того, менеджерам нужна статистика по найденным и исправленным ошибкам – это хороший

инструмент контроля проекта. Все это изображено на рис. 8.2. Чтобы справиться с этим потоком информации и обеспечить необходимые в работе, удобные сервисы, существует специальный класс программных средств – *средства контроля ошибок*.



Рисунок 8.2 – Обработка ошибок

Как правило, описание ошибки в системе контроля ошибок имеет следующие основные атрибуты:

- ответственного за ее проверку – тестировщика, который ее нашел и который проверяет, что исправления, сделанные разработчиком, действительно устраняют ошибку;
- ответственного за ее исправление – разработчика, которому ошибка отправляется на исправление;
- состояние, например, ошибка найдена, ошибка исправлена, ошибка закрыта, ошибка вновь проявилась и т.д.

Этот список существенно дополняется в различных программных средствах контроля ошибок, но это основные атрибуты.

Использование этих систем давно стало общей практикой в разработке ПО, наравне со средствами версионного контроля и многими иными инструментами. Они включают в себя:

- базу данных для хранения ошибок;
- интерфейс к этой базе данных для внесения новых ошибок и задания их многочисленных атрибутов, для просмотра ошибок на основе различных фильтров – например, все найденные ошибки за последний месяц, все ошибки, за которые отвечает данный разработчик и т.д.;

- сетевой доступ, так как проекты все чаще оказываются распределенными;
- программный интерфейс для возможностей программной интеграции таких систем с другим ПО, поддерживающим разработку ПО (например, со средствами непрерывной интеграции – они могут автоматически вносить в базу данных найденные при автоматическом прогоне тестов ошибки).

Очень важным при работе с ошибками оказываются различные отчеты.

Рефакторинг

Качество кода программного продукта во многом определяется тем, насколько трудно или легко вносить изменения в код, а также тем, насколько доступен код для понимания. Созданное программное приложение может выполнять требуемые функции, но иметь проблемы с внесением изменений или пониманием созданного кода. В этом случае такое ПО нельзя назвать качественным, так как на этапе его сопровождения могут возникнуть проблемы с его модификацией при изменении пользовательских требований.

Для улучшения качества кода программных приложений применяют рефакторинг [2, 17]. По определению М. Фаулера, рефакторинг определяется как «процесс изменения программной системы таким образом, что её внешнее поведение не изменяется, а внутренняя структура улучшается». Следовательно, в процессе проектирования для создания высококачественного программного продукта необходимо не только обеспечить выполнение функциональных требований, но и нефункциональных, в частности, удобства сопровождения, что предполагает возможность и простоту внесения изменений в код, а также возможность легкого понимания созданного кода.

Некачественный дизайн кода определяется по ряду признаков [12]:

- жесткость – характеристика программы, затрудняющей внесение изменений в код;
- хрупкость – свойство программы повреждаться во многих местах при внесении единственного изменения;

- косность характеризуется тем, что код содержит части, которые могли бы оказаться полезными в других системах, но усилия и риски, сопряженные с попыткой отделить эти части кода от оригинальной системы, слишком велики;
- ненужная сложность характеризуется тем, что программа содержит элементы, которые не используются в текущий момент;
- ненужные повторения, которые состоят в повторяющихся фрагментах кода в программе;
- непрозрачность, которая характеризует трудность кода для понимания.

Для создания качественного дизайна кода целесообразно применять некоторые принципы и паттерны проектирования ПО [2, 12].

Принцип единственной обязанности определяет, что у класса должна быть только одна причина для изменения. Из этого принципа следует, что классу целесообразно поручать только одну обязанность.

Принцип открытости/закрытости определяет, что программные сущности (классы, модули, функции) должны быть открыты для расширения, но закрыты для модификации.

Принцип подстановки Лисков определяет, что должна быть возможность вместо базового класса подставить любой его подтип.

Принцип инверсии зависимостей определяет два положения:

- модули верхнего уровня не должны зависеть от модулей нижнего уровня. И те, и другие должны зависеть от абстракций;
- абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций.

Принцип разделения интерфейсов определяет, что клиенты должны знать только об абстрактных интерфейсах, обладающих свойством сцепленности.

Паттерны проектирования предлагают универсальные, проверенные практикой решения. В [2] приведен обширный перечень паттернов. Среди общего списка паттернов следует выделить те, которые целесообразно применять при гибкой разработке

программного обеспечения. Это паттерны Команда, Стратегия, Фасад, Посредник, Одиночка, Фабрика, Компоновщик, Наблюдатель, Абстрактный сервер/адаптер/шлюз, Заместитель и Шлюз, Посетитель и Состояние.

Использование паттернов при разработке позволяет создавать программное обеспечение, которое легко модифицировать и сопровождать.

Следует отметить, что для проведения рефакторинга необходимо иметь надежные тесты, которые обеспечивают контроль соблюдения функциональных требований при улучшении дизайна кода программного обеспечения.

Ключевые термины

ITU (International Telecommunication Union) – комитет по стандартизации телекоммуникационных протоколов и интерфейсов.

ISO (International Organization for Standardization) – организация по содействию развития стандартизации, а также смежных видов деятельности в мире с целью обеспечения международного обмена товарами и услугами.

ETSI (European Telecommunications Standards Institute) – независимая, некоммерческая организация по стандартизации в телекоммуникационной промышленности в Европе.

SEI (Software Engineering Institute) – организация по стандартизации в области программной инженерии, выработке критериев для сертификации надежных и зрелых компаний.

IEEE (Institute of Electrical and Electronics Engineers) – международная некоммерческая ассоциация специалистов в области техники и разработки стандартов по радиоэлектронике и электротехнике.

OMG (Object Management Group) – организация по разработке и продвижению объектно-ориентированных технологий и стандартов.

Тестирование – проверка соответствия между реальным поведением программы и ее ожидаемым поведением в специально заданных, искусственных условиях.

Рефакторинг – процесс изменения программной системы без изменения её внешнего поведения для улучшения внутренней структуры кода.

Краткие итоги

Вопросами стандартизации на международном уровне занимаются такие организации, как: ITU (International Telecommunication Union), ISO (International Organization for Standardization), ETSI (European Telecommunications Standards Institute) и комитетов SEI (Software Engineering Institute), IEEE (Institute of Electrical and Electronics Engineers) и OMG (Object Management Group). Все эти комитеты и организации включают программную инженерию в сферу своей деятельности. Стандартизация качества ПО прежде всего проявляется в вопросах тестирования программного обеспечения и стандартизации процессов его создания. К методам обеспечения качества ПО относятся: наладка качественного процесса разработки, формальные методы доказательства корректности программ, исследование и анализ динамических свойств ПО, обеспечение качества кода и тестирование. Тестирование представляет собой процесс проверки соответствия между реальным поведением программы и ее ожидаемым поведением в специально заданных, искусственных условиях. Существуют различные виды тестирования: модульное, интеграционное, системное, регрессионное, нагрузочное, стрессовое и приемочное. Для улучшения качества кода программных приложений применяют рефакторинг.

Контрольные вопросы

1. Назовите стандарты качества ISO.
2. Назовите стандарты телекоммуникационных протоколов и интерфейсов ITU.
3. Национальные и международные комитеты по стандартизации.
4. Стандартизация качества ПО.
5. Качество программного обеспечения.
6. Дайте характеристику методам обеспечения качества ПО.
7. Тестирование программных средств.
8. Виды тестирования программных средств.
9. Поясните назначение модульного тестирования.
10. Поясните назначение интеграционного тестирования.
11. Поясните назначение системного тестирования.
12. Поясните назначение регрессионного тестирования.